

LOCAL PROCESSES

A much stronger physical architectural boundary is the local process. A local process is typically created from the command line or an equivalent system call. Local processes run in the same processor, or in the same set of processors within a multicore, but run in separate address spaces. Memory protection generally prevents such processes from sharing memory, although shared memory partitions are often used.

Most often, local processes communicate with each other using sockets, or some other kind of operating system communications facility such as mailboxes or message queues.

Each local process may be a statically linked monolith, or it may be composed of dynamically linked deployment components. In the former case, several monolithic processes may have the same components compiled and linked into them. In the latter, they may share the same dynamically linked deployment components.

Think of a local process as a kind of uber-component: The process consists of lower-level components that manage their dependencies through dynamic polymorphism.

The segregation strategy between local processes is the same as for monoliths and binary components. Source code dependencies point in the same direction across the boundary, and always toward the higher-level component.

For local processes, this means that the source code of the higher-level processes must not contain the names, or physical addresses, or registry lookup keys of lower-level processes. Remember that the architectural goal is for lower-level processes to be plugins to higher-level processes.

Communication across local process boundaries involve operating system calls, data marshaling and decoding, and interprocess context switches, which are moderately expensive. Chattiness should be carefully limited.

SERVICES

The strongest boundary is a service. A service is a process, generally started from the command line or through an equivalent system call. Services do not depend on their physical location. Two communicating services may, or may not, operate in the same

physical processor or multicore. The services assume that all communications take place over the network.

Communications across service boundaries are very slow compared to function calls. Turnaround times can range from tens of milliseconds to seconds. Care must be taken to avoid chatting where possible. Communications at this level must deal with high levels of latency.

Otherwise, the same rules apply to services as apply to local processes. Lower-level services should “plug in” to higher-level services. The source code of higher-level services must not contain any specific physical knowledge (e.g., a URI) of any lower-level service.

CONCLUSION

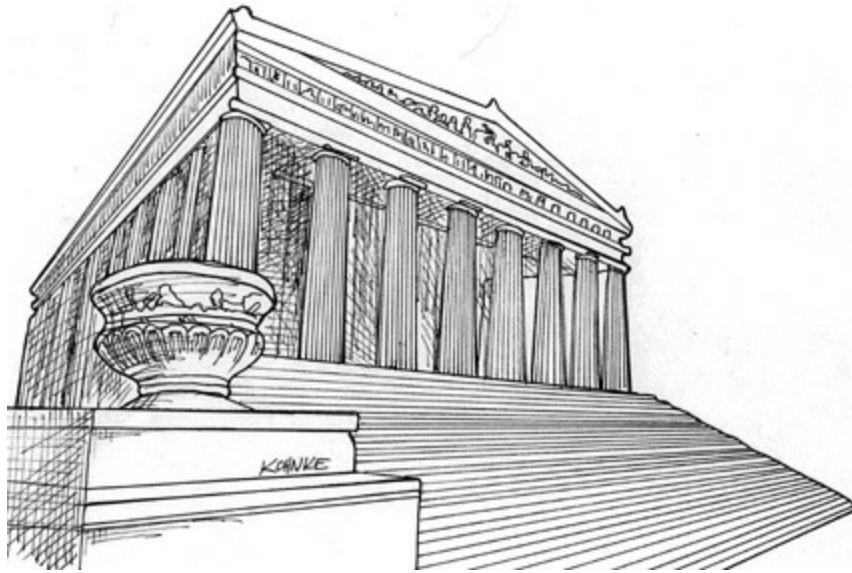
Most systems, other than monoliths, use more than one boundary strategy. A system that makes use of service boundaries may also have some local process boundaries. Indeed, a service is often just a facade for a set of interacting local processes. A service, or a local process, will almost certainly be either a monolith composed of source code components or a set of dynamically linked deployment components.

This means that the boundaries in a system will often be a mixture of local chatty boundaries and boundaries that are more concerned with latency.

1. Static polymorphism (e.g., generics or templates) can sometimes be a viable means of dependency management in monolithic systems, especially in languages like C++. However, the decoupling afforded by generics cannot protect you from the need for recompilation and redeployment the way dynamic polymorphism can.
2. Although static polymorphism is not an option in this case.

19

POLICY AND LEVEL



Software systems are statements of policy. Indeed, at its core, that's all a computer program actually is. A computer program is a detailed description of the policy by which inputs are transformed into outputs.

In most nontrivial systems, that policy can be broken down into many different smaller statements of policy. Some of those statements will describe how particular business rules are to be calculated. Others will describe how certain reports are to be formatted. Still others will describe how input data are to be validated.

Part of the art of developing a software architecture is carefully separating those policies from one another, and regrouping them based on the ways that they change. Policies that change for the same reasons, and at the same times, are at the same level and belong together in the same component. Policies that change for different

reasons, or at different times, are at different levels and should be separated into different components.

The art of architecture often involves forming the regrouped components into a directed acyclic graph. The nodes of the graph are the components that contain policies at the same level. The directed edges are the dependencies between those components. They connect components that are at different levels.

Those dependencies are source code, compile-time dependencies. In Java, they are `import` statements. In C#, they are `using` statements. In Ruby, they are `require` statements. They are the dependencies that are necessary for the compiler to function.

In a good architecture, the direction of those dependencies is based on the level of the components that they connect. In every case, low-level components are designed so that they depend on high-level components.

LEVEL

A strict definition of “level” is “the distance from the inputs and outputs.” The farther a policy is from both the inputs and the outputs of the system, the higher its level. The policies that manage input and output are the lowest-level policies in the system.

The data flow diagram in [Figure 19.1](#) depicts a simple encryption program that reads characters from an input device, translates the characters using a table, and then writes the translated characters to an output device. The data flows are shown as curved solid arrows. The properly designed source code dependencies are shown as straight dashed lines.

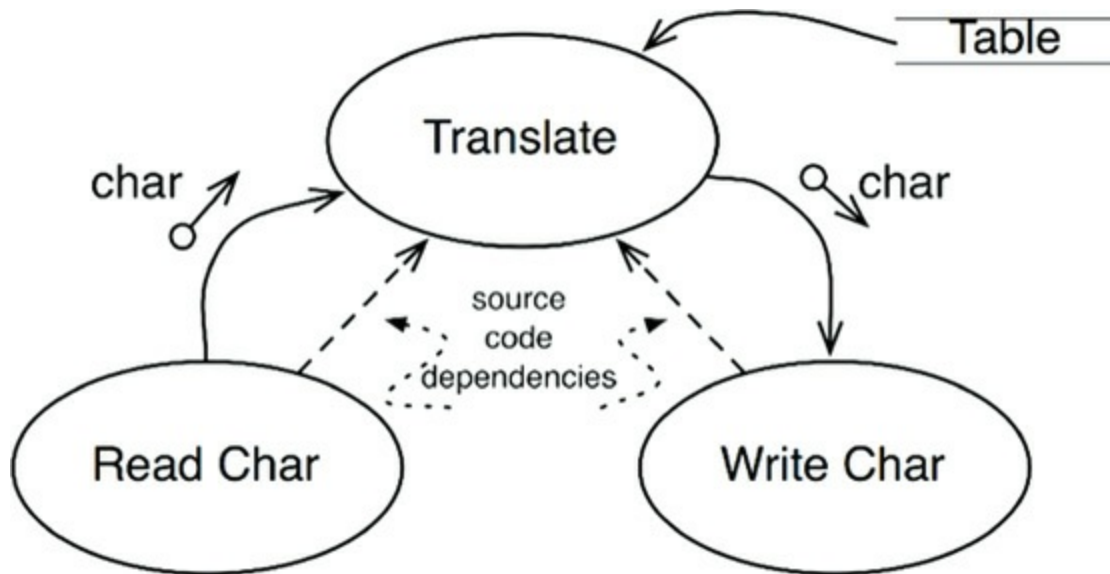


Figure 19.1 A simple encryption program

The `Translate` component is the highest-level component in this system because it is the component that is farthest from the inputs and outputs.¹

Note that the data flows and the source code dependencies do not always point in the same direction. This, again, is part of the art of software architecture. We want source code dependencies to be decoupled from data flow and *coupled to level*.

It would be easy to create an incorrect architecture by writing the encryption program like this:

[Click here to view code image](#)

```
function encrypt() {  
    while(true)  
        writeChar(translate(readChar()));  
}
```

This is incorrect architecture because the high-level `encrypt` function depends on the lower-level `readChar` and `writeChar` functions.

A better architecture for this system is shown in the class diagram in [Figure 19.2](#). Note the dashed border surrounding the `Encrypt` class, and the `CharWriter` and `CharReader` interfaces. All dependencies crossing that border point inward. This unit is the highest-level element in the system.

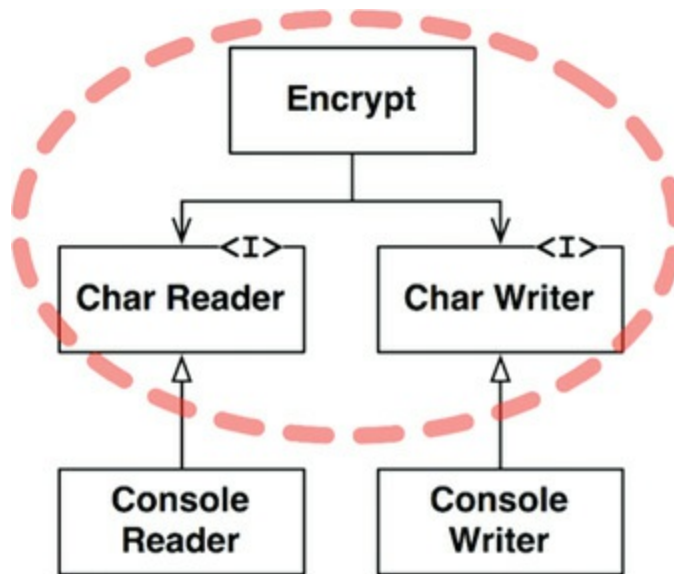


Figure 19.2 Class diagram showing a better architecture for the system

`ConsoleReader` and `ConsoleWriter` are shown here as classes. They are low level because they are close to the inputs and outputs.

Note how this structure decouples the high-level encryption policy from the lower-level input/output policies. This makes the encryption policy usable in a wide range of contexts. When changes are made to the input and output policies, they are not likely to affect the encryption policy.

Recall that policies are grouped into components based on the way that they change. Policies that change for the same reasons and at the same times are grouped together by the SRP and CCP. Higher-level policies—those that are farthest from the inputs and outputs—tend to change less frequently, and for more important reasons, than lower-level policies. Lower-level policies—those that are closest to the inputs and outputs—tend to change frequently, and with more urgency, but for less important reasons.

For example, even in the trivial example of the encryption program, it is far more likely that the IO devices will change than that the encryption algorithm will change. If the encryption algorithm does change, it will likely be for a more substantive reason than a change to one of the IO devices.

Keeping these policies separate, with all source code dependencies pointing in the direction of the higher-level policies, reduces the impact of change. Trivial but urgent changes at the lowest levels of the system have little or no impact on the higher, more important, levels.